ParER –Evoluzione Architettura

Introduzione CDC per “Strangler Pattern” su Database

26 febbraio 2026

ENGINEERING

Architettura AS-IS

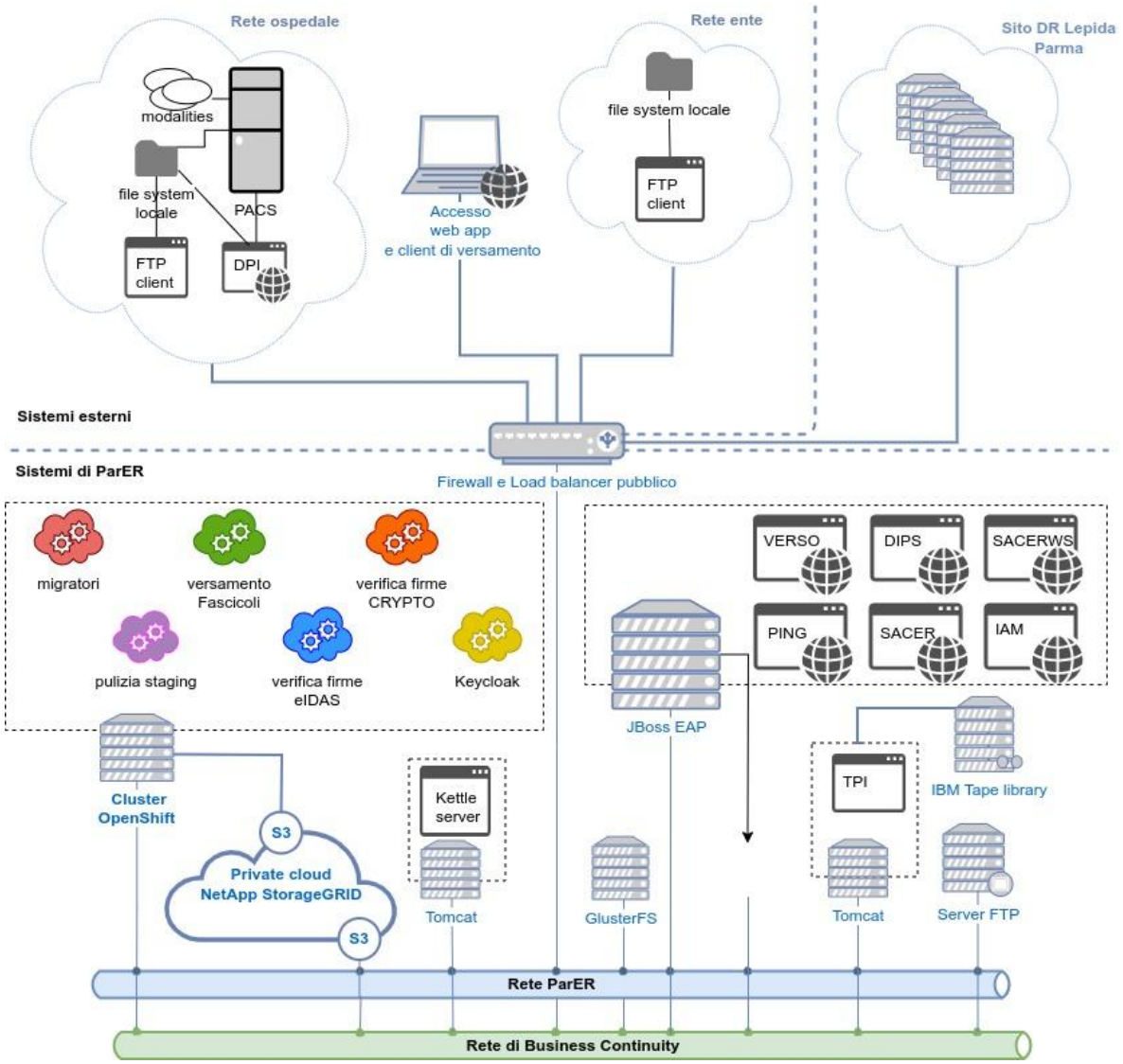
JBoss 7.x / 8.x
Cluster 8 nodi standalone

OpenShift 4.14

Google Cloud Storage

Oracle RAC 19c
Cluster 2 nodi

Tomcat 9.0.74



© Engineering

Evoluzione architetturale

Il gruppo di lavoro Eng interno al ParER ha proposto, nel 2023, l'avvio di una nuova progettazione dell'architettura software per garantire ulteriore sostenibilità della **gestione delle risorse** e per migliorare le **performance** del sistema vista la mole di dati gestita.
La proposta consiste nel **passaggio da un sistema a microservizi con base dati monolitica ad un sistema a microservizi con base dati distribuita.**

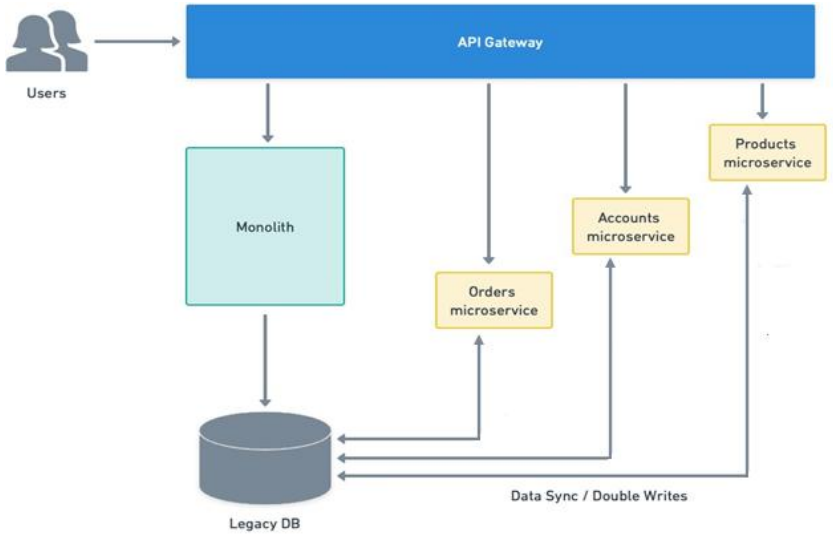
L'approccio a **monolite** ha delle implicazioni dirette nella gestione delle applicazioni, delle **tecnologie** utilizzate, dell'intera **infrastruttura**, nella gestione della **banca dati**
Un **esempio** di limite dell'attuale approccio a monolite è la difficoltà di accesso ai metadati versati (vedi dati specifici) che comporta una non fruibilità / **limite** di alcune **ricerche "avanzate**"

Inoltre **Sacer** è un "**accentratore**"; in pratica, tutta la base di conoscenza sui metadati versati (che sono l'informazione / documento che identificano l'oggetto versato) risiede su tale contesto applicativo (servizi esposti, funzionalità UI, base dati, etc.)

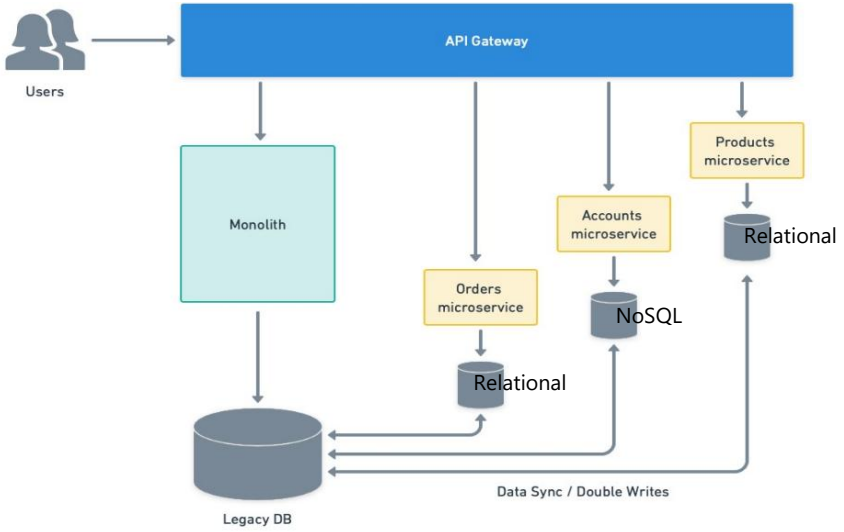
L'approccio proposto di decentramento ed integrazione è la strada da seguire nell'evoluzione di architetture SoA altamente distribuite che implica:

- Scomposizione delle componenti software con basi dati "proprietarie"
- Integrazione / condivisione delle informazioni (documenti) che può essere mediata da sistemi event driven (es.: Kafka)

AS IS - Sistema a microservizi con base dati monolitica



TO BE - Sistema a microservizi con base dati distribuita



© Engineering

DOMINI FUNZIONALI SACER

Schema sintetico dei 6 domini e delle principali responsabilità

CONFIGURAZIONE

Ruoli e abilitazioni
Tipologie / profili
Policy di conservazione

PRE-ACQUISIZIONE

Preparazione / raccolta
Controlli preliminari

VERSAMENTO

Acquisizione pacchetti
Metadati / validazioni

ARCHIVIAZIONE

Conservazione / storage
Accesso e recupero

RICERCHE

Query e filtri
Esportazioni / report

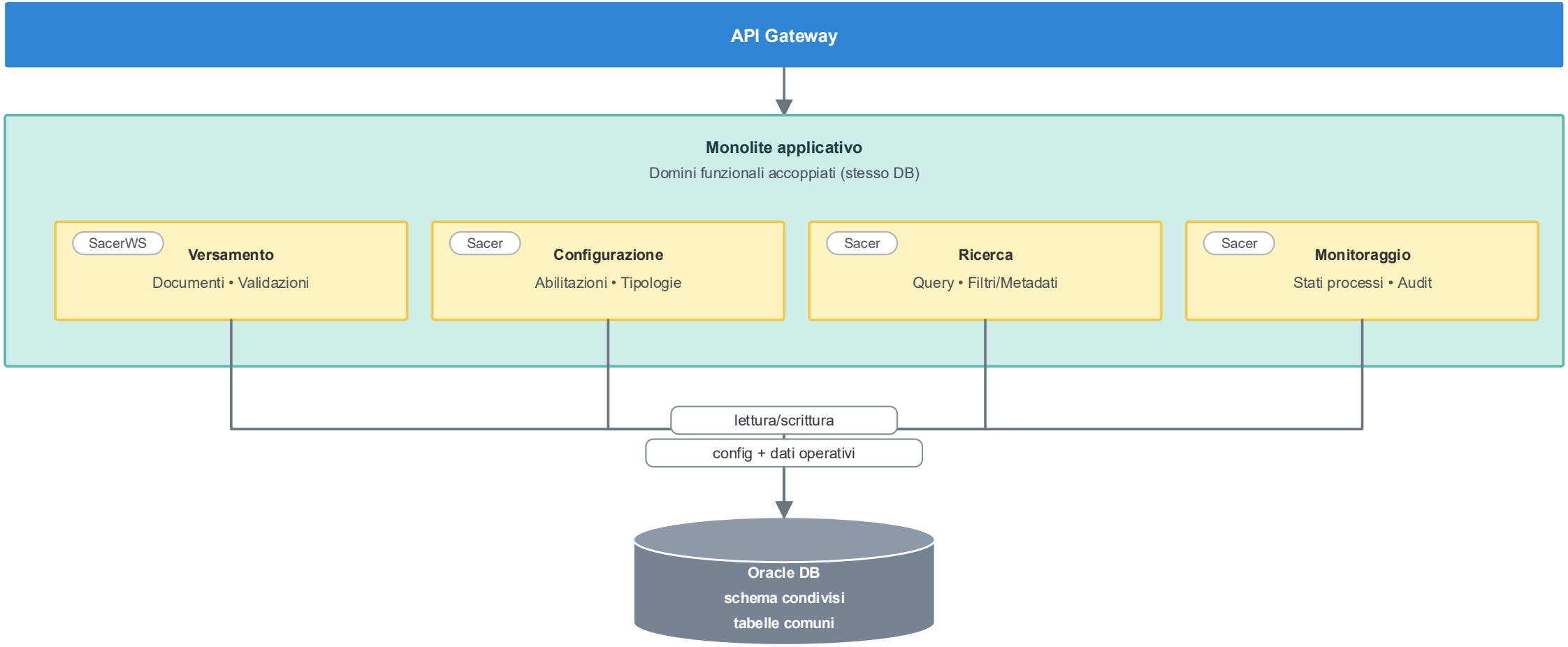
MONITORAGGIO

Stati di processo
Audit log / anomalie

© Engineering

Architettura Attuale – Applicazione Sacer

Domini funzionali accoppiati tramite Database Oracle condiviso

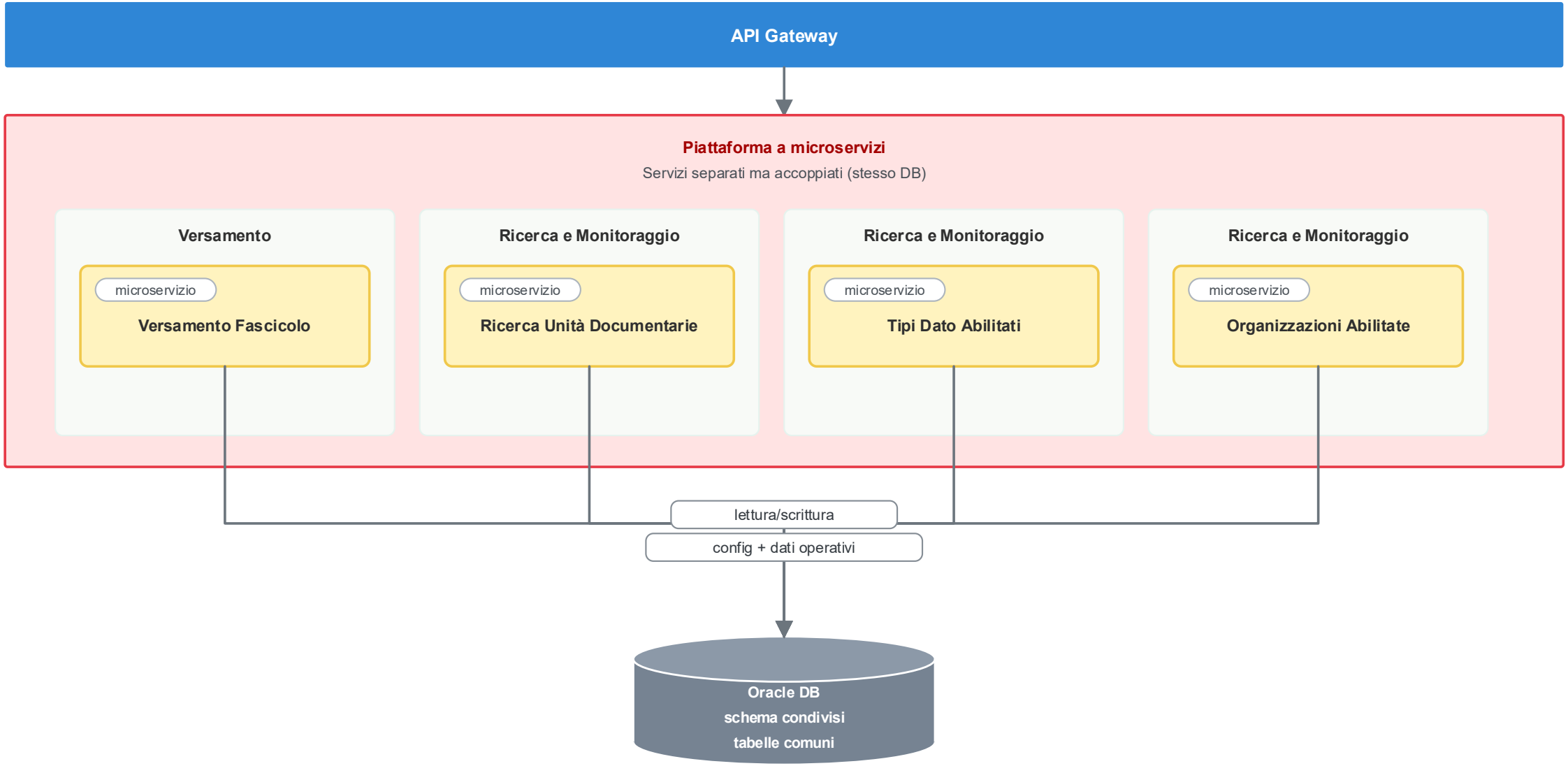


Elevato accoppiamento: il Database è il punto centrale di integrazione tra i domini

© Engineering

Architettura Attuale – Servizi SACER

Servizi indipendenti (runtime separato) — Database Oracle centralizzato



Accoppiamento sul DB: microservizi separati ma integrazione centrale su Database

© Engineering

Dove siamo oggi

Stiamo già evolvendo, ma abbiamo un limite strutturale.

- Oggi SacER sta evolvendo come un sistema a microservizi, dove:
  - I servizi sono separati
  - I dati non lo sono, questo crea accoppiamento forte
- Tutta la «conoscenza» vive ancora in un'unica base dati centrale
  - È un sistema che sta crescendo troppo!

© Engineering

Il problema reale

Il «collo di bottiglia» principale non è applicativo, è nei dati.

- Ogni dominio dipende dalla stessa base dati:
  - Tutti i domini leggono e scrivono sulla stessa base dati
  - L'integrazione tra domini avviene a livello di database
  - Il sistema è un accentramento di responsabilità
  - Il database è il punto centrale di coordinamento dell'intero sistema
- Oggi la base dati è il punto di integrazione tra i domini
  - Questo non è un dettaglio tecnico, è un vincolo architetturale!

© Engineering

Cosa succede se non interveniamo

Non è un problema urgente. È un problema inevitabile!

- Crescita dei dati → crescita della complessità
  - Ogni dominio NON scala in base al proprio carico
- Ridotta autonomia dei domini
  - Non posso evolvere uno schema in autonomia
- Maggior difficoltà nel fare evoluzione funzionale e aumento dei rischi su «change» evolutivi
  - Una modifica allo schema dati impatta potenzialmente più domini
  - Ogni «change» diventa potenzialmente sistemico
  - Un'ottimizzazione per un dominio può degradarne un altro
  - L'ottimizzazione diventa un compromesso globale

© Engineering

Il principio guida

Ogni dominio deve essere responsabile dei propri dati.

- Separare i servizi non basta
- Serve separare anche la base dati
- Nessun dominio deve accedere direttamente ai dati di un altro

© Engineering

La meta finale (architettura a regime)

Non è una riscrittura. È una migrazione controllata.

- Domini con DB proprietari
- Integrazione via eventi e API
- «Read model» dedicati per query SQL e NoSQL
- SacER progressivamente ridotto

© Engineering

Il metodo: perché Strangler Pattern

Niente «big bang».

- Non fermiamo il sistema
- Non facciamo migrazione massiva
- Non duplichiamo tutto per sempre

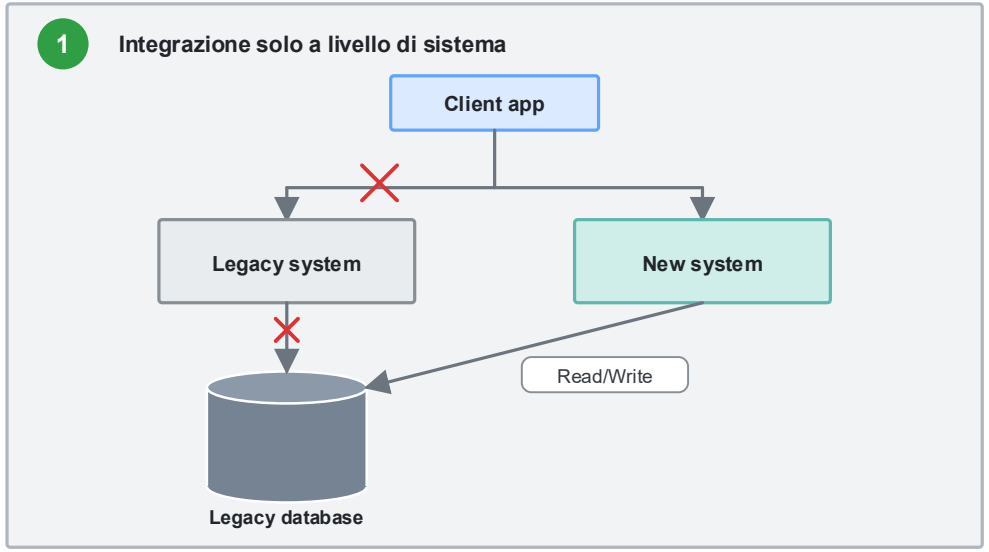
Strangler Pattern: sostituzione progressiva per domini.

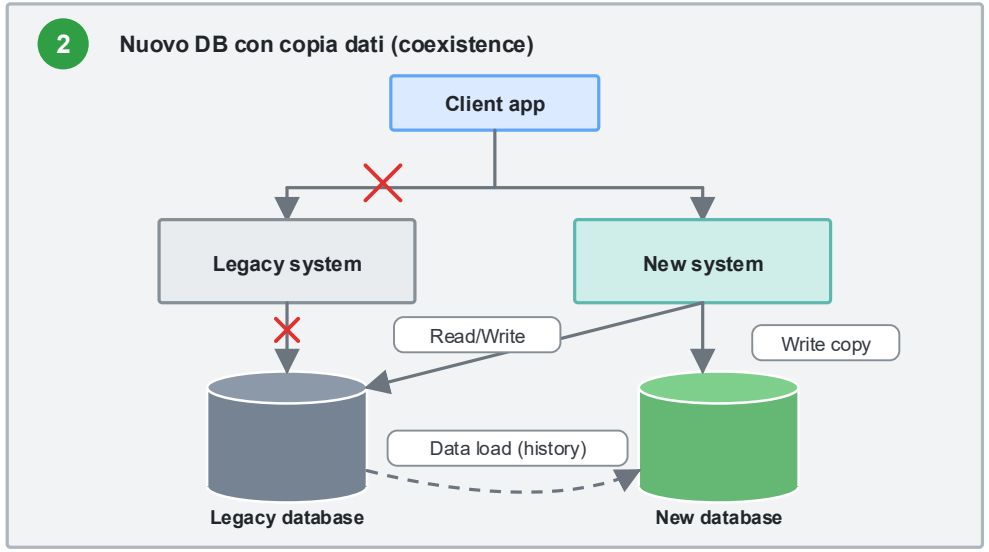
- Ogni dominio viene isolato, migrato e reso autonomo
- Il resto continua a funzionare
- Le componenti legacy si disattivano progressivamente

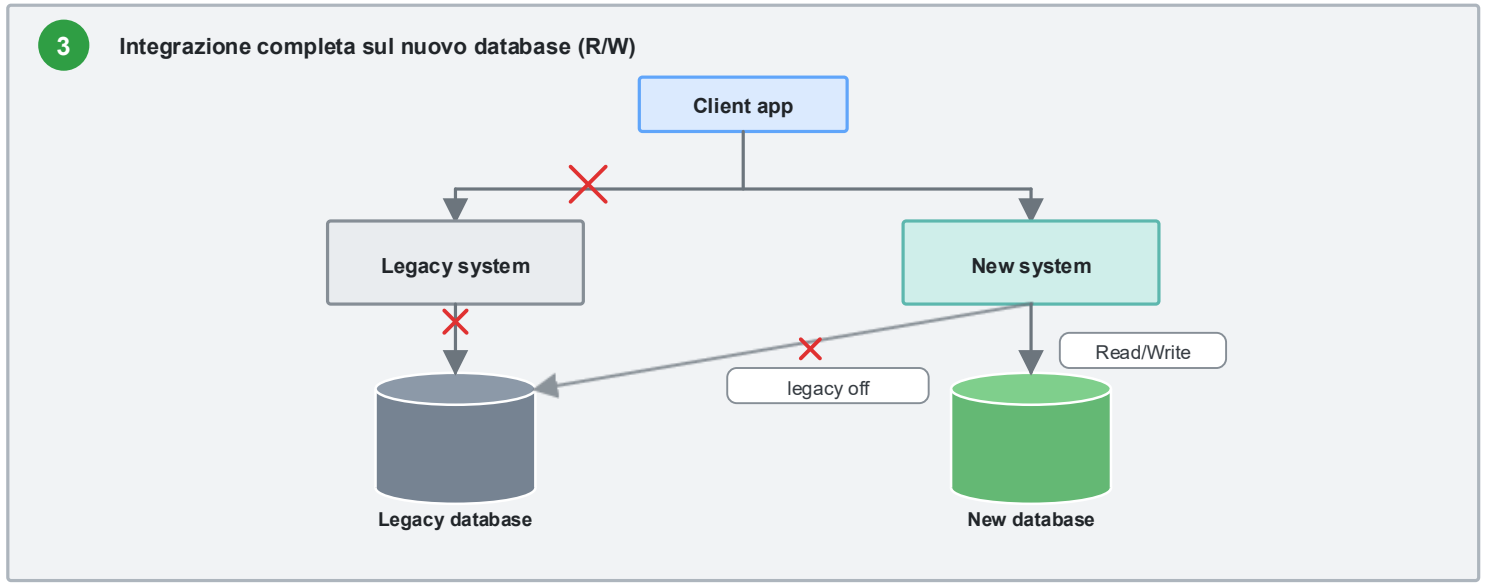
© Engineering

Strangler Pattern — Percorso di integrazione (3 step)

I sistemi Legacy tipicamente dipendono da un database centralizzato.
Con il passare del tempo, un database centralizzato può diventare complesso da evolvere e gestire, a causa delle molte dipendenze.
Applicando il pattern "Stragler Fig" sul database con un approccio graduale è possibile transitare da un sistema legacy a un nuovo sistema riducendo al minimo gli impatti.







Nota: quale strategia attuare quando serve sincronizzazione/streaming dei dati tra domini?

© Engineering

Il ruolo del CDC (Change-Data-Capture)

Come facciamo a migrare un dominio senza rompere gli altri?

- Introducendo un meccanismo di replica controllata dei dati
  - CDC
  - Event streaming

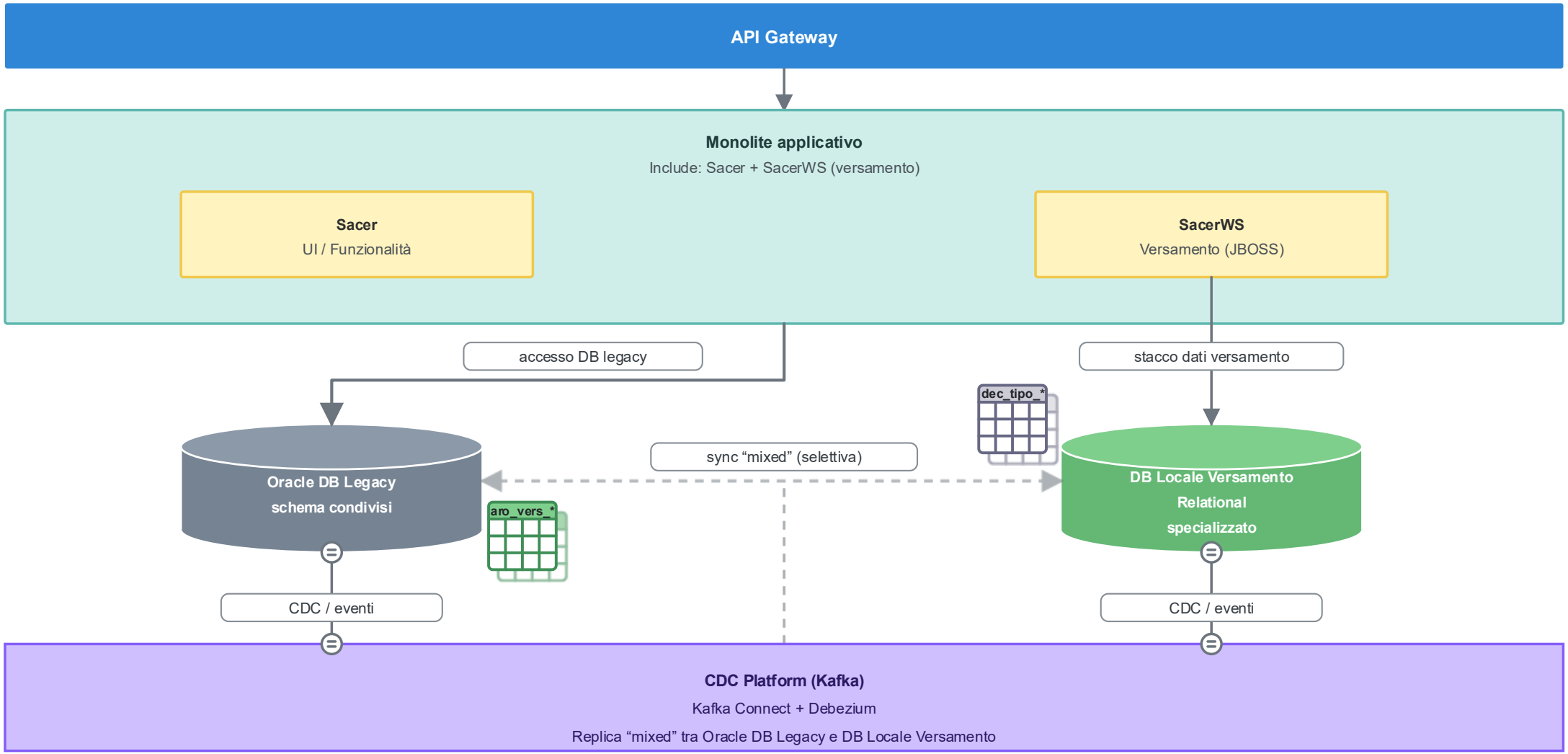
Cosa garantisce il CDC:

- Continuità operativa
- Sincronizzazione controllata
- Riduzione del rischio
- Possibilità di rollback

© Engineering

Evoluzione – Introduzione CDC per “Strangler Pattern” sul Versamento

Nel panorama in continua evoluzione dell'elaborazione dei dati in tempo reale, il Change Data Capture (CDC) è la tecnica per mantenere i sistemi di dati sincronizzati. Debezium, combinato con Kafka Connect, offre una soluzione robusta per lo streaming delle modifiche ai database direttamente in Apache Kafka. Alla fine, si ottiene una potente pipeline CDC pronta per catturare e trasmettere le modifiche da Oracle a Kafka, abilitando l'integrazione dei dati in tempo reale.



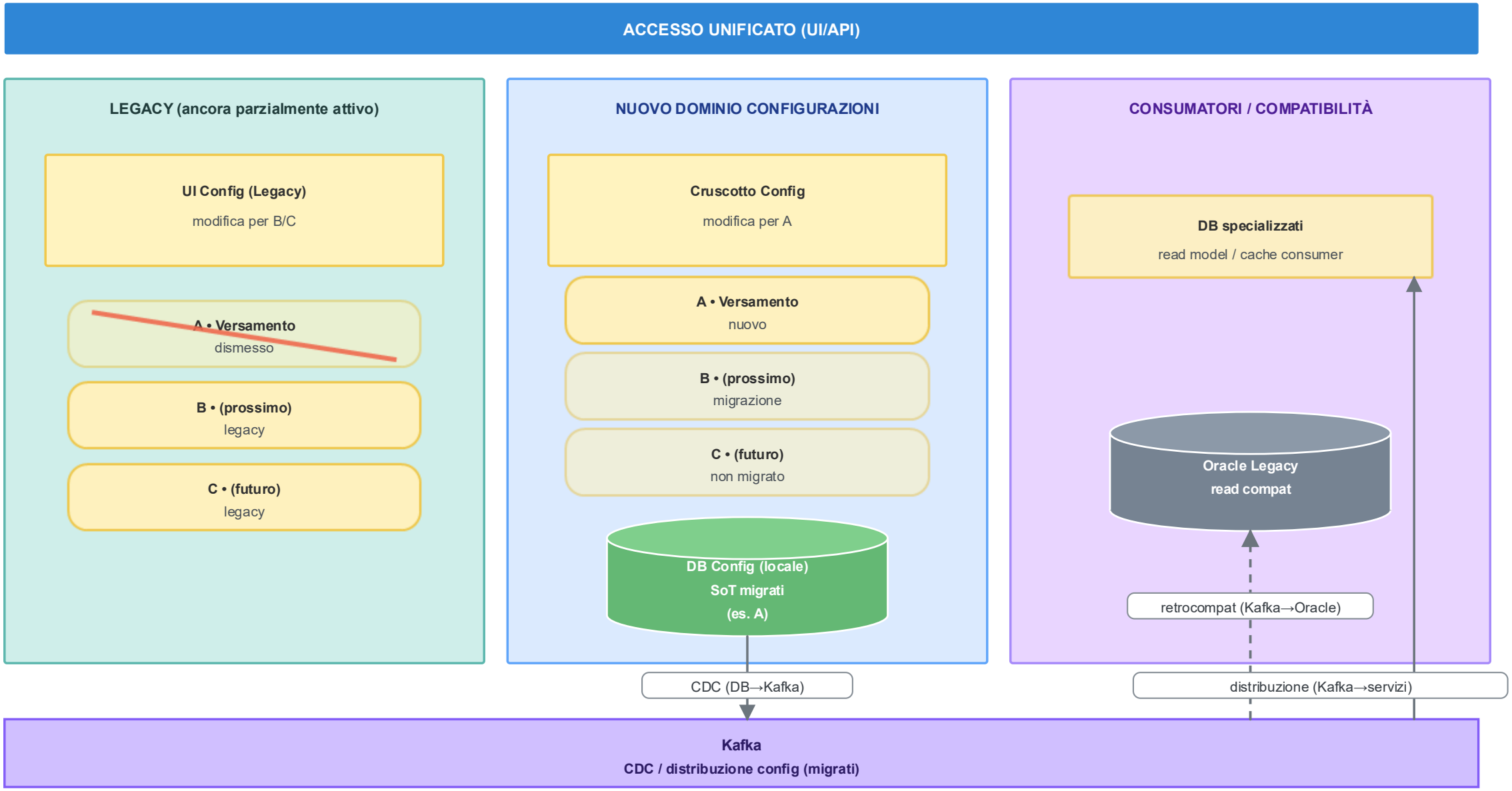
Strangler pattern: si introduce un DB locale per il Versamento mantenendo Oracle in coesistenza (CDC Kafka con replica e write-back selettivi)

Nota: quale strategia attuare per eliminare completamente la dipendenza dei dati dal DB Legacy?

© Engineering

Migrazione Configurazioni (Strangler Fig) • Fase 1

Coesistenza per sottodomini: alcuni si modificano sul nuovo Cruscotto, altri restano (temporaneamente) sul legacy

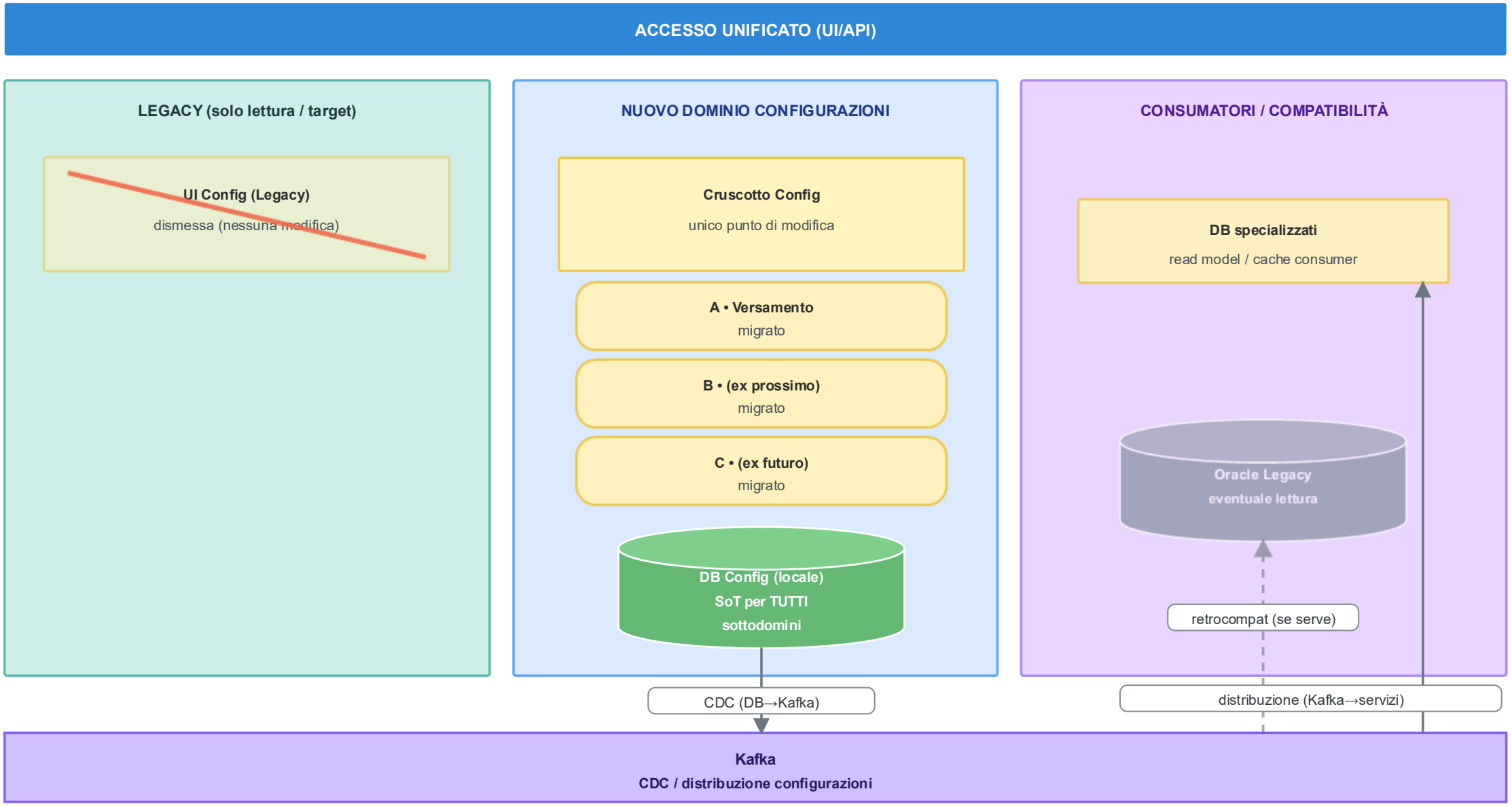


© Engineering

Strangler: migra 1 sottodominio alla volta → nuovo SoT → CDC ai consumer → legacy attivo solo per non migrati

Migrazione Configurazioni (Strangler Fig) • Fase 2

Completata la migrazione: tutte le configurazioni sono gestite dal nuovo dominio; il legacy è dismesso come punto di modifica

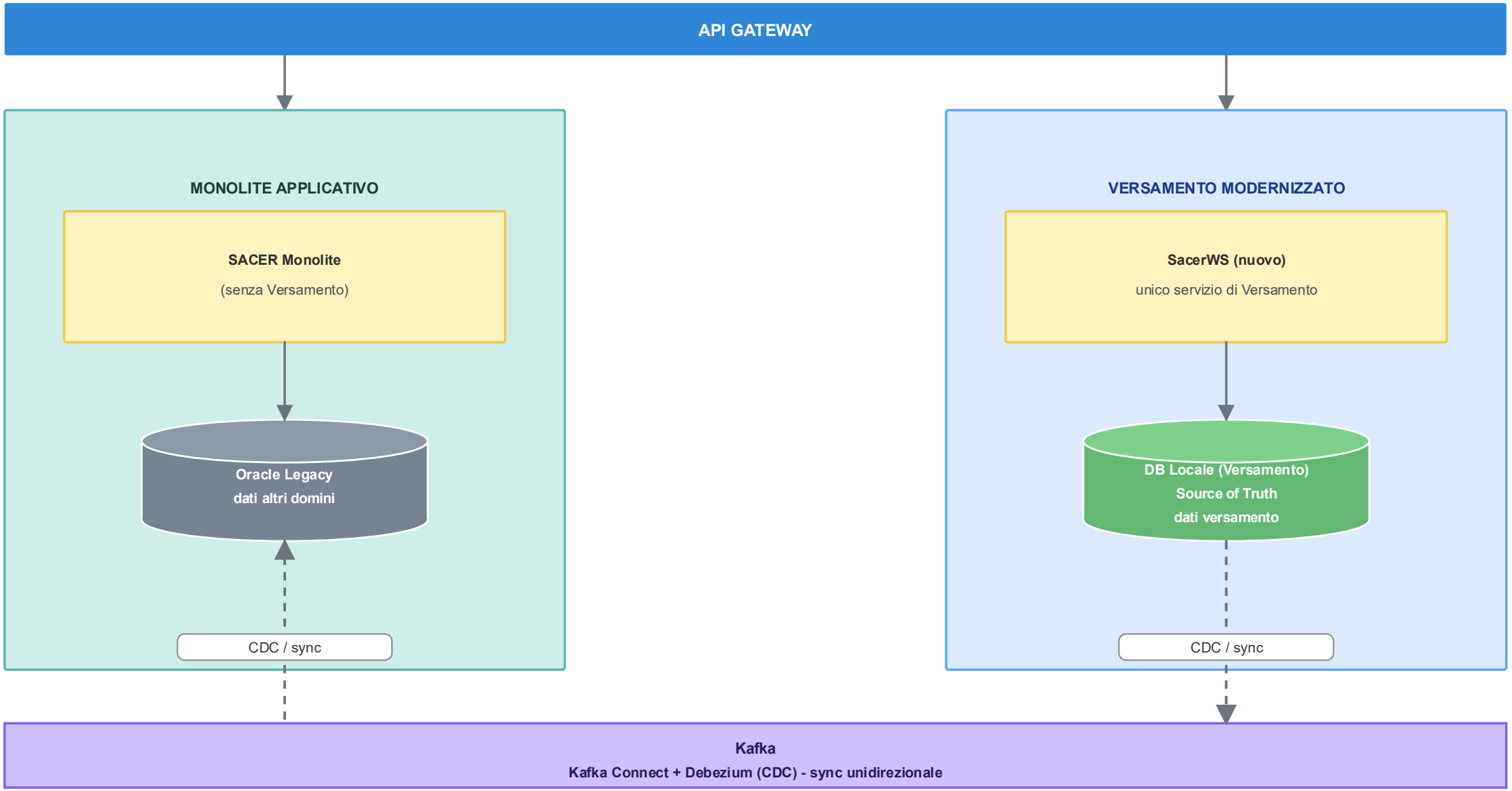


© Engineering

Fine strangler: legacy dismesso come UI di modifica → nuove config gestite solo sul nuovo dominio → CDC distribuisce verso chi consuma

Versamento isolato, replica su DB Legacy

Kafka Connect + Debezium: Dati Versamento DB locale -> DB Legacy

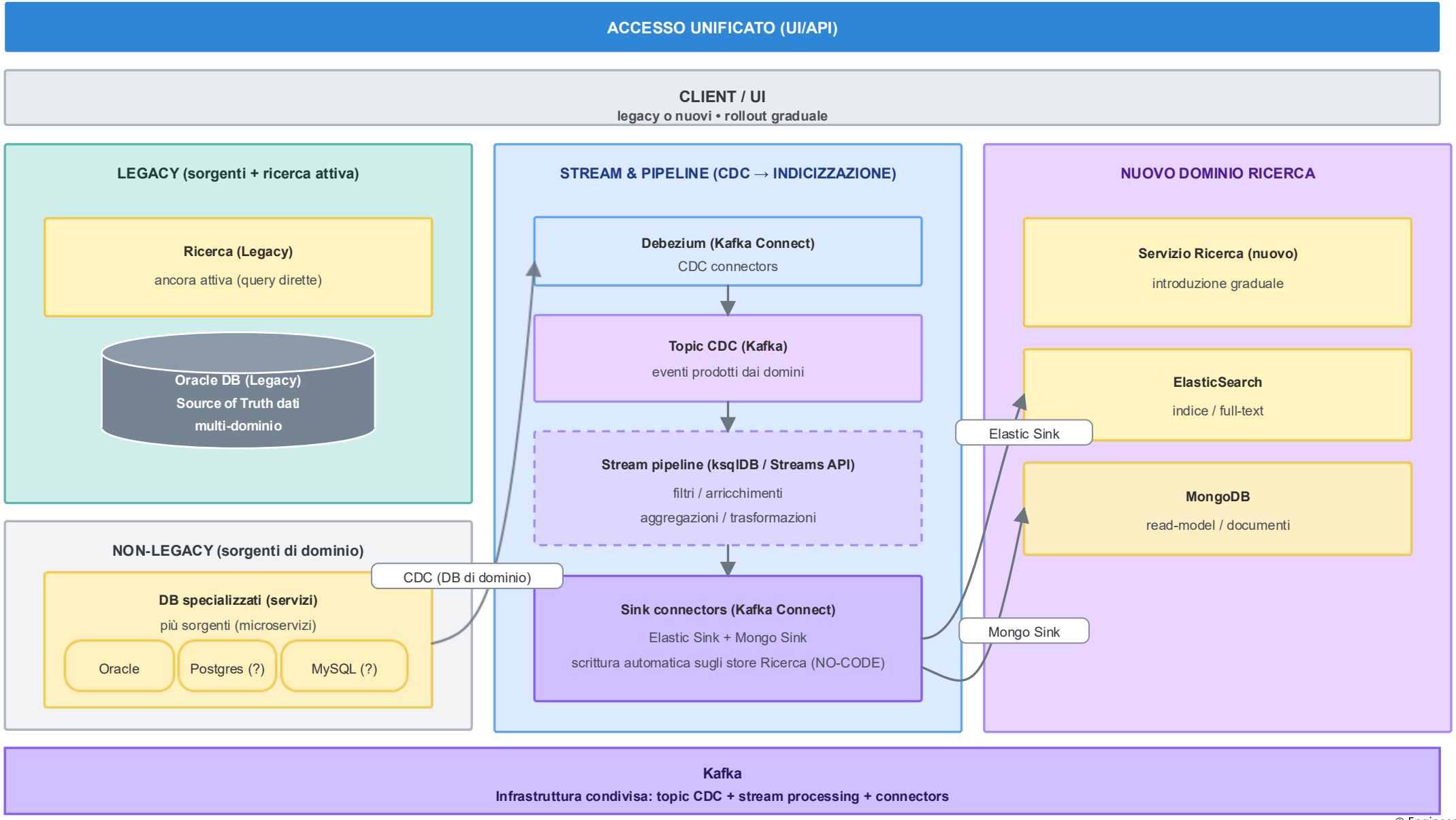


© Engineering

Finché altri domini restano su DB Legacy: CDC necessario per allineare Dati di Versamento

DOMINIO RICERCA (Strangler Fig) • Fase 1

La ricerca legacy resta attiva; si introduce la pipeline stream per alimentare Elastic + Mongo (NO-CODE)

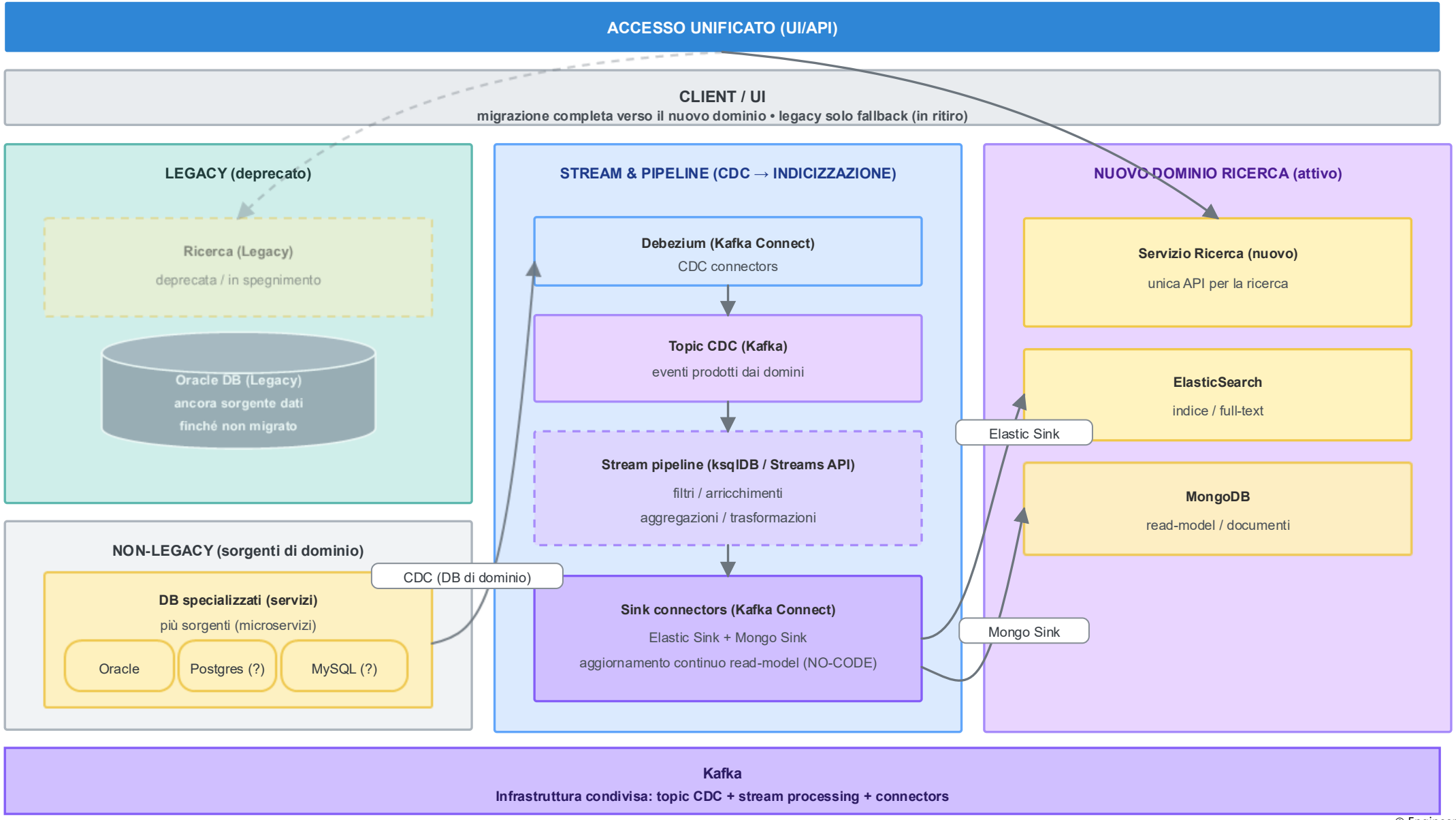


© Engineering

Fase 1: senza spegnere la ricerca legacy, si abilita una pipeline a stream che aggiorna automaticamente Elastic/Mongo (NO-CODE).

DOMINIO RICERCA (Strangler Fig) • Fase 2

Cutover progressivo: il client punta al nuovo servizio; la ricerca legacy viene deprecata e poi spenta

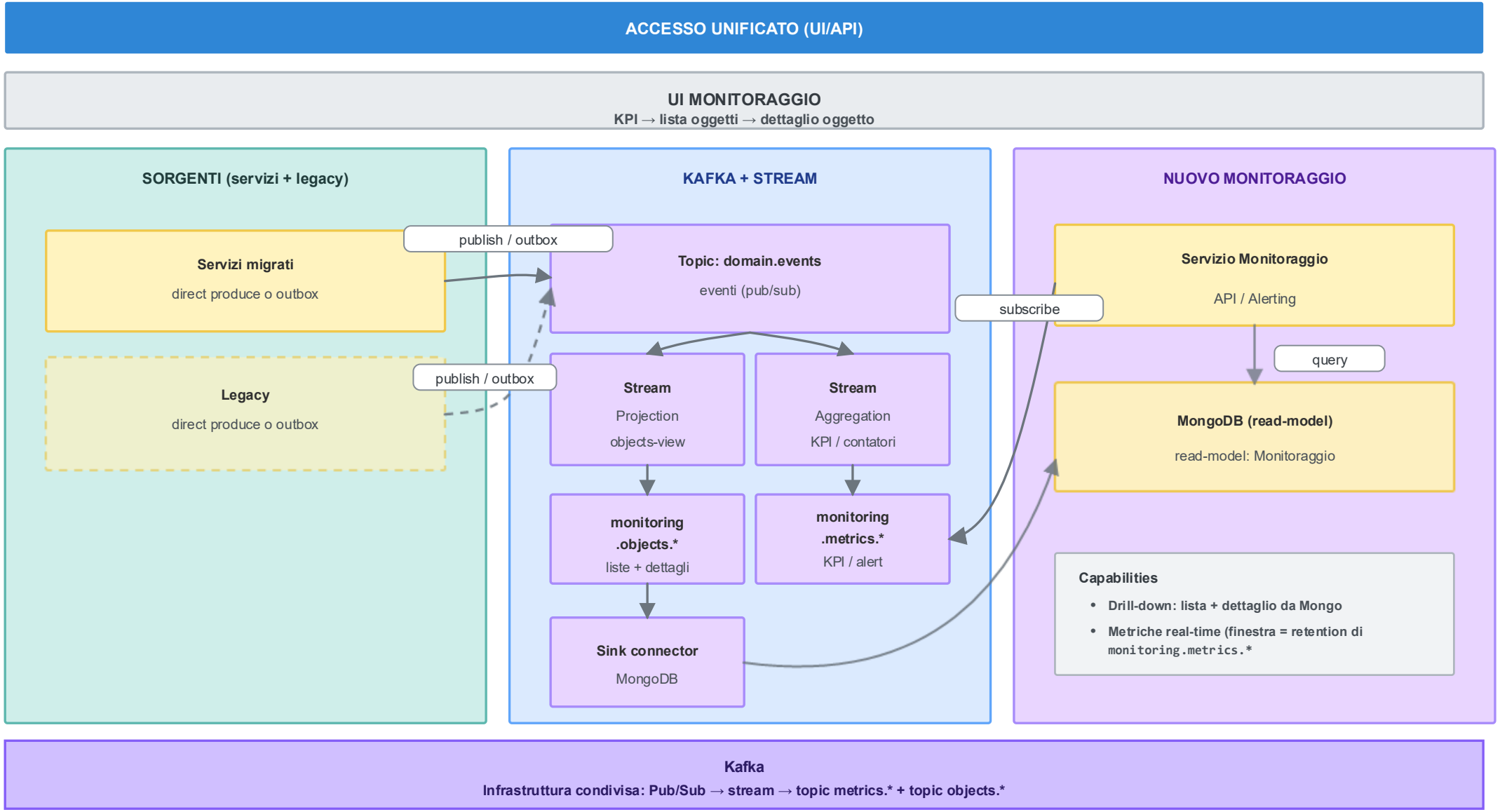


© Engineering

Fase 2: il traffico passa al nuovo Servizio Ricerca; la componente legacy resta solo come fallback temporaneo e viene poi dismessa. La pipeline Kafka continua ad alimentare Elastic + Mongo.

DOMINIO MONITORAGGIO (Strangler Fig) • Fase 1

Pub/Sub + Outbox Pattern: eventi → KPI real-time + objects-view per drill-down (read-model su Mongo) senza query su DB operativi

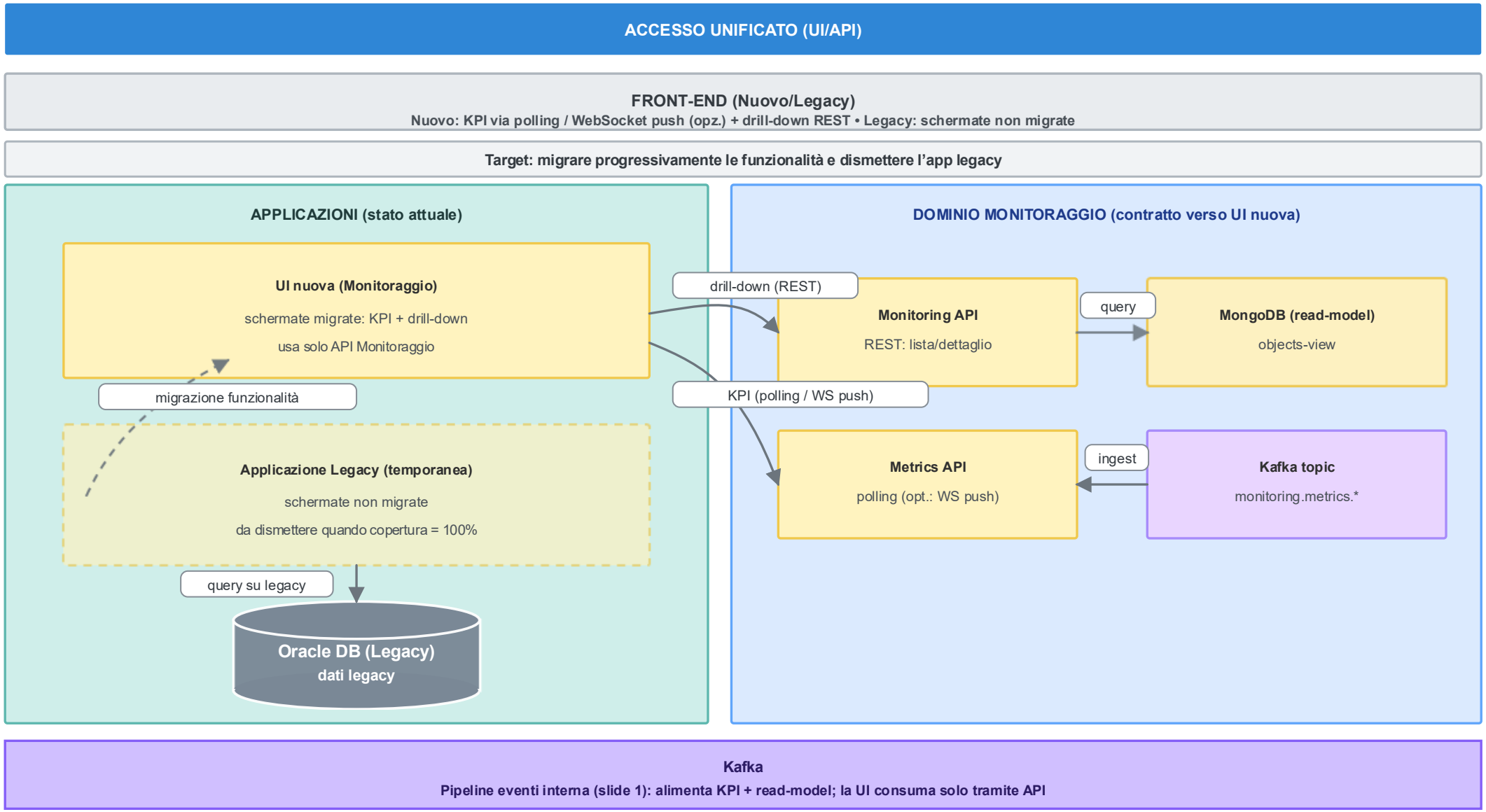


© Engineering

Fase 1: metrics (subscribe diretto) + objects-view (materializzato su Mongo via sink) abilitano drill-down senza query su DB operativi.

DOMINIO MONITORAGGIO (Strangler Fig) • Fase 2

Oggi: UI nuova + applicazione legacy in parallelo • Target: consolidare sul nuovo e dismettere la legacy app



© Engineering

La UI nuova si appoggia a un read-model (MongoDB) e a API dedicate, per consentire migrazione graduale e dismissione della legacy app.

INTEGRAZIONE TRA DOMINI • Modelli: CDC / Outbox / Direct Pub/Sub / Read-model

Integrazione event-driven: ogni dominio sceglie il modello in base a coerenza, replica dati e natura dello stream

Contratto: integrazione via eventi + proiezioni/repliche (no accesso diretto ai DB di altri domini)

OUTBOX — evento di dominio “transazionale”
CDC — replica/sync dati
DIRECT PUB/SUB — stream/metriche
READ-MODEL — proiezioni per query

DOMINI (servizi) • modello di integrazione

Versamento (nuovo SacerWS)
Eventi: outbox → event log • Sync: CDC verso legacy/repliche
OUTBOX CDC

Configurazioni (nuovo dominio)
Eventi: outbox → event log • Distribuzione config: CDC (projection)
OUTBOX CDC

Ricerca (nuovo dominio)
Consuma eventi → materializza read-model (Elastic + Mongo). Eventi prodotti: opzionali.
READ-MODEL

Monitoraggio funzionale (nuovo dominio)
Metriche/KPI: direct pub/sub → event log • Drill-down via REST su read-model
DIRECT PUB/SUB

READ-MODEL / STORAGE (proiezioni)

DB Versamento (SoT)
fonte verità del dominio Versamento

DB Config (SoT)
fonte verità del dominio Config

ElasticSearch (indice) — Ricerca
query full-text / filtri

MongoDB (read-model) — Ricerca
proiezioni/documenti per ricerca

MongoDB (read-model) — Monitoraggio
drill-down REST (dettagli/oggetti)

topic metrics.*
retention variabile (KPI / contatori / alert)

Kafka
Event log per integrazione tra domini: topic + retention + replay (outbox / CDC / direct pub/sub alimentano il bus)

© Engineering

Roadmap “Strangler Fig” per domini – sintesi

Kafka come abilitatore: CDC → Event Log → Read-model specializzati → dismissione funzionalità legacy

Kafka (CDC + Event Log)

abilita migrazione “a sottodomini” senza interruzioni

Versamento

DB per servizio

replica verso legacy finché serve

legacy versamento spento

Configurazioni

Cruscotto + DB Config

SoT nel nuovo dominio

replica verso legacy/servizi

Ricerca

UI + ElasticSearch + MongoDB

read-model via stream

ricerca legacy spenta

Monitoraggio

integrazione via API REST

metriche real-time

stream aggregation

read-model via stream

Esito complessivo

Meno accoppiamento	Migrazione sicura	Dati “pronti”
niente DB come “bus”	per sottodomini, step-by-step	read-model + real-time KPI

Target: architettura a domini con integrazione event-driven; il legacy resta solo dove serve e viene dismesso progressivamente

© Engineering

Roadmap e Obiettivo Finale

Fase 1:

- Introduzione backbone eventi
  - Apache Kafka ✓
  - Kafka Connect ✓
  - KsqlDB ✓
  - Schema Registry ✓
  - Debezium

Fase 2:

- Migrazione primo dominio (pilota)

…

Obiettivo: dotarsi di un'architettura flessibile che può evolvere e scalare nel tempo.

© Engineering